Lecture Notes on CS231N

Yiwei Chen

Zhejiang University

Starting from March 26, 2025

Last updated: April 30, 2025

Contents

Lecture 1: History of CV and Introduction to CNNs

- **ImageNet:** Annual competition for image classification, started in 2010.
- **Convolutional Neural Networks (CNNs):** Introduced by Yann LeCun in 1998, CNNs are a type of neural network designed for processing structured grid data, such as images. CNNs show great performance in image classification tasks.

Lecture 2: Image Classification Pipeline

2.1 Goal and Attempts

1. **Goal:**

   The task in Image Classification is to predict a single label (or a distribution over labels as shown here to indicate our confidence) for a given image. Images are 3-dimensional arrays of integers from 0 to 255, of size Width x Height x 3. The 3 represents the three color channels Red, Green, Blue.

2. **Attempts:**

   - Find edges, then corners: does not work well.
   - Use large datasets with labels.

2.2 Classifiers:

1. **K-Nearest Neighbors (KNN):**

   - *Description:* When $K = 1$, Find the closest image in the dataset to the input image (Nearest Neighbors (NN)).
   - *Distance metric:*

     (a) L1(Mahanttan) Distance: a squared distance metric.

     $$d(x, y) = \sum_i |x_i - y_i| \tag{2.1}$$

     (b) L2(Euclidean) Distance: a squared distance metric.

     $$d(x, y) = \sqrt{\sum_i (x_i - y_i)^2} \tag{2.2}$$

     Rotating the coordinate system changes the L1 distance but not the L2 distance.

   - *Performance:*

     Training time: $O(1)$, as there is nothing to do.

     Prediction time: $O(N)$, which is inefficient.

   - K-Nearest Neighbors (KNN):

     Description:

     When $K = 1$, the classifier is too sensitive to noise.

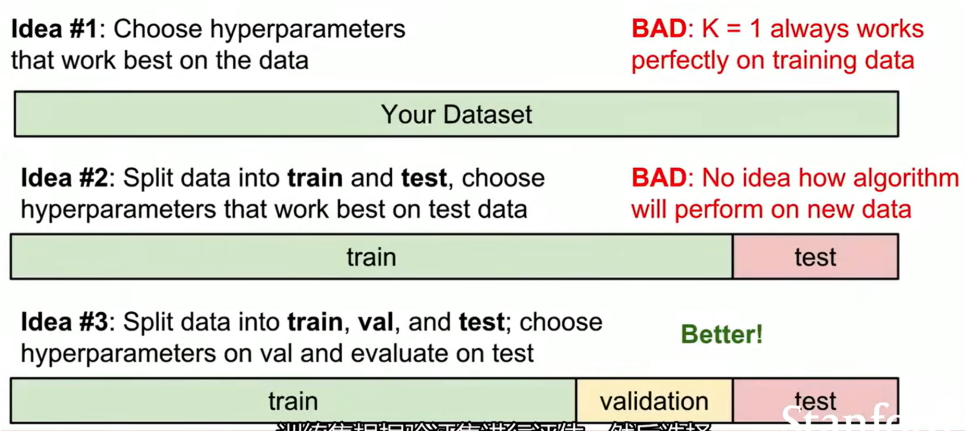
     Instead of copying the label of the closest image, take the majority vote of the K closest images.

   - *hyperparameters*(超参数):

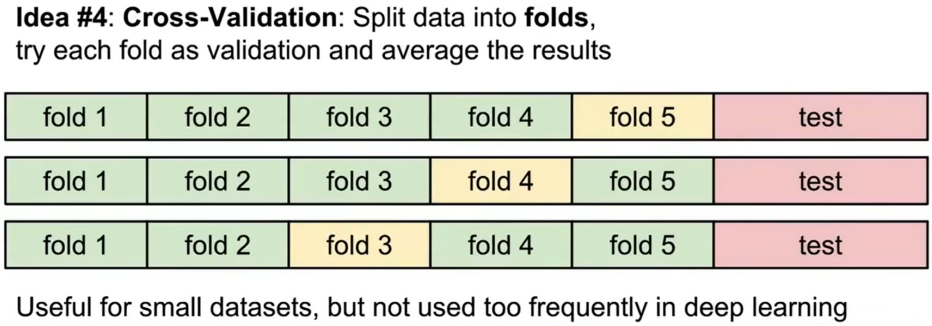
     Choices about the model that are not learned from the data, e.g., K in KNN.

     To set hyperparameters:

     - Never use the test set to set hyperparameters.
     - Splitting data into train and test is not enough.
     - **The better idea:** Splitting the training set into training set, validation set, and test set.



– **The common idea:** Cross-validation(交叉验证).



- *Pros and Cons:*

  Actually, KNN on image is never used:

  - Very slow at test time.

  - Distance-metrics on pixels are not informative.

  - Curse of dimensionality: as the number of dimensions increases, the distance between points becomes less meaningful.

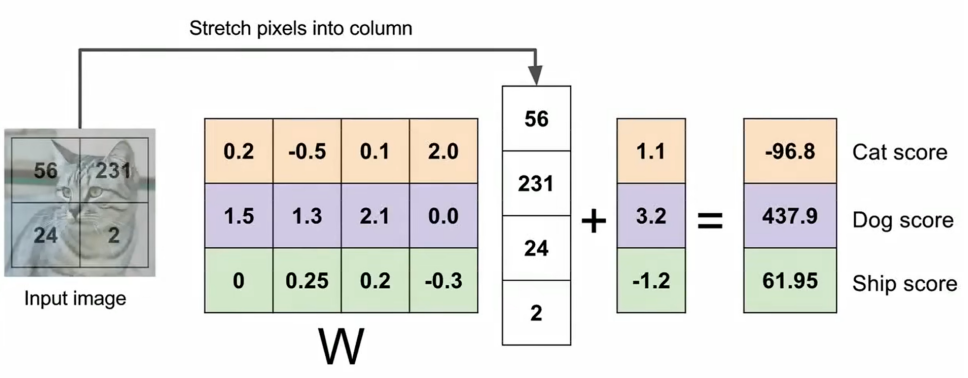
2. **Linear Classifier:**

- *Description:* A linear classifier makes its predictions based on a linear predictor function combining a set of weights with the feature vector.

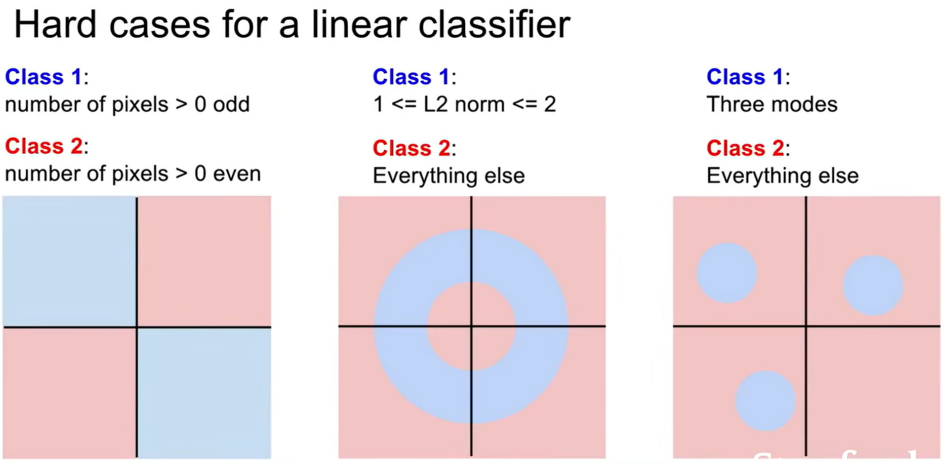
$$f(x, W) = Wx + b \tag{2.3}$$

where x is the input image and w is the weight vector, b is the bias term.

And it is important to center the data before applying the linear classifier, namely image data preprocessing.



- *Hard cases:*


Hard cases for a linear classifier
Class 1:
number of pixels > 0 odd
Class 2:
number of pixels > 0 even
Class 1:
1 <= L2 norm <= 2
Class 2:
Everything else
Class 1:
Three modes
Class 2:
Everything else

Lecture 3: Loss Functions and Optimization

3.1 Loss Functions

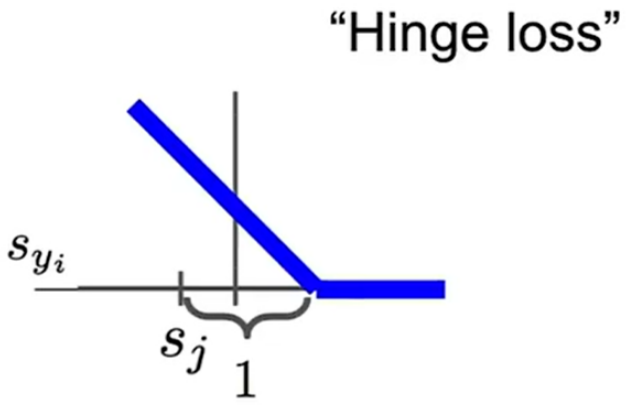
1. **Multiple SVM loss:**

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_{y_i} - s_j + 1 & \text{otherwise} \end{cases}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) = \sum_{j \neq y_i} \max(0, w_j^T x_j - w_{y_i}^T x_{y_i} + 1) \qquad \text{(SVM-Loss)}$$

where y_i is the label for class i, $s = f(x_i, W)$ is the score for class i, where wj is the j-th row of W **reshaped as a column**.

And this kind of loss function is called **Hinge Loss**.



$$L = \frac{1}{N} \sum_{i=1}^{N} L_i \tag{3.1}$$

For the first training, the W is randomly initialized, so the loss is close to C - 1, where C is the number of classes, which can be used to check the correctness of the implementation.

2. **regularization:**

Furthermore, we can add a regularization term to the loss function to prevent overfitting:

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda R(W) = \frac{1}{N} \sum_{i=1}^{N} \sum_{j \neq y_i} \max(0, f(x_i, W)_j - f(x_i, W)_{y_i} + \Delta) + \lambda R(W) \tag{3.2}$$

Here Δ actually does not matter. It turns out that this hyperparameter can safely be set to $\Delta = 1.0$ in all cases.

The hyperparameters Δ and λ seem like two different hyperparameters, but in fact they both control the same tradeoff: The tradeoff between the data loss and the regularization loss in the objective.

Therefore, the exact value of the margin between the scores is in some sense meaningless because the weights can shrink or stretch the differences arbitrarily. Hence, the only real tradeoff is how large we allow the weights to grow (through the regularization strength λ).

In common use:

- **L2 regularization:** $R(W) = ||W||^2$
- L1 regularization: $R(W) = ||W||_1$
- Elastic net: $R(W) = \beta||W||^2 + ||W||_1$
- Max norm regularization: might see later
- Dropout: might see later
- Fansier: Batch normalization, stochastic depth, etc.

3. **Softmax Classifier(Multinomial Logistic Regression):**

   scores = unormalized log probabilities of the classes.

$$P(y_i|x_i) = \frac{e^{s_{y_i}}}{\sum_{j=1}^{C} e^{s_j}} \quad \text{where } C \text{ is the number of classes} \tag{3.3}$$

$$L_i = -\log P(y_i|x_i) = -\log \frac{e^{s_{y_i}}}{\sum_{j=1}^{C} e^{s_j}} = -s_{y_i} + \log \sum_{j=1}^{C} e^{s_j} \tag{3.4}$$

   For Softmax, even though the right score is much higher than all the other scores, the loss is still not zero, which is different from SVM.

   Here, it's actually a cross-entropy loss function, which is defined as follows:

$$H(p, q) = -\sum_{x} p(x) \log q(x) \qquad \text{(Cross-entropy)}$$

   where p is the "true" distribution and q is the predicted distribution.

   For Softmax, $q = e^{s_j} / \sum_{k=1}^{C} e^{s_k}$, and the "true" distribution is a one-hot vector(p = [0,...,1,...,0] contains a single 1 at the y_i-th position).

3.2 Optimization:

Strategy: follow the slope

So first we need to compute the gradient of the loss function with respect to the weights W.

The method of finite differences($(f(W+h)-f(W))/h$) is bad because of large amount of computation, dont't use it to train but to **check the correctness** of the implementation.

step size/learning rate: a hyperparameter that controls how much to change the model in response to the estimated gradient.

1. **Derivative of the loss function:**

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W) \tag{3.5}$$

   The scores of each class are influenced by the rows of W corresponding to the classes.

Hence, the gradient of the loss function with respect to the weights W should focus on the rows.(assuming W is (C * D))

- **For Hinge Loss:**

$$L_i = \sum_{j \neq y_j} [\max(0, w_j^T x_i - w_{y_i}^T x_i + \Delta)]$$

where w_j is the j-th row of W reshaped as a column.

Taking the gradient with respect w_{y_i}

$$\nabla_{w_{y_i}} Li = -(\sum_{j \neq y_j} 1(w_j^T x_j - w_{y_i}^T x_{y_i} + \Delta > 0))x_i \tag{3.6}$$

where 1 is the indicator function that is one if the condition inside is true or zero otherwise.
For the other rows where $j \neq y_i$ (for every j) the gradient is:

$$\nabla_{w_j} Li = 1(w_j^T x_j - w_{y_i}^T x_{y_i} + \Delta > 0)x_i \tag{3.7}$$

- **For Softmax Loss:**

$$L_i = -\log P(y_i|x_i) = -\log \frac{e^{s_{y_i}}}{\sum_{j=1}^{C} e^{s_j}} = -s_{y_i} + \log \sum_{j=1}^{C} e^{s_j}$$

Taking the gradient with respect w_{y_i}

$$\nabla_{w_{y_i}} Li = -x_i + \frac{e^{s_{y_i}}}{\sum_{j=1}^{C} e^{s_j}} x_i = -x_i + P(y_i|x_i)x_i \tag{3.8}$$

For the other rows where $j \neq y_i$ (for every j) the gradient is:

$$\nabla_{w_j} Li = \frac{e^{s_j}}{\sum_{k=1}^{C} e^{s_k}} x_i = P(j|x_i)x_i \tag{3.9}$$

2. **Stochastic Gradient Descent(on-line gradient descent):**

   Full sum computation is too expensive, so we can use mini-batch(32/64/128) gradient descent.

   Every iteration, we randomly sample a mini-batch of N training examples from the training set, and compute the gradient of the loss function with respect to the weights W using only this mini-batch.

   SGD technically refers to using a single example at a time to evaluate the gradient.

   However, you will hear people use the term SGD even when referring to mini-batch gradient descent (i.e. mentions of MGD for "Minibatch Gradient Descent", or BGD for "Batch gradient descent" are rare to see), where it is usually assumed that mini-batches are used.

   MGD: using a small batch of examples to evaluate the gradient.

   BGD: using the entire training set to evaluate the gradient.

3.3 Image features:

Some times, only using the raw pixel values is not enough, we need to extract some features from the image.

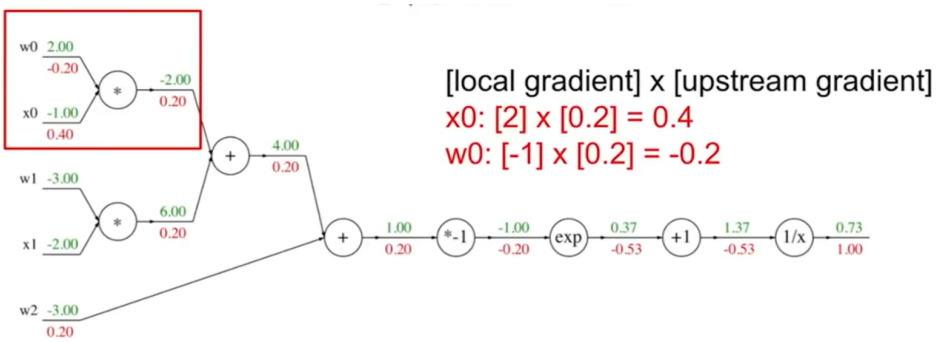
For example:

- color histogram(直方图): a representation of the distribution of colors in an image.
- HOG(Histogram of Oriented Gradients): a feature descriptor focusing on the edges and contours of objects in an image.
- Bag of Words: a representation of an image as a collection of local features, where each feature is represented by a visual word.

Lecture 4: Backpropagation and Neural Networks

4.1 Backpropagation

1. **Computation Graph:**



Computation Graph Example: $f(x) = \frac{1}{1+e^{-(\omega_0 x_0 + \omega_1 x_1 + \omega_2)}}$

Along the edges of the graph, we do **forward** computations to get the scores of the classes, and get the output label.

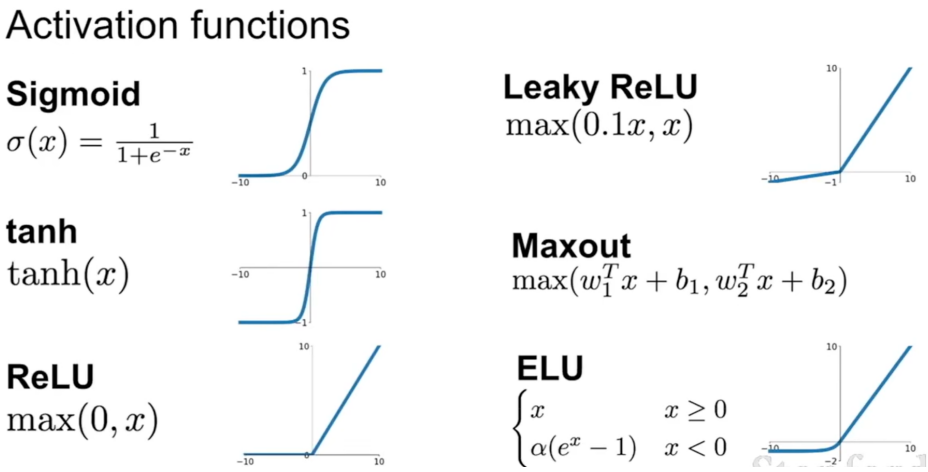
And **backward** computations to get the gradients of the loss function with respect to the weights W and the input x.

2. **Backpropagation**

Sigmoid funtion:

$$\sigma(x) = \frac{1}{1+e^{-x}} \tag{4.1}$$

It is a common activation function used in neural networks, especially in the output layer for binary classification tasks.



Activation Functions

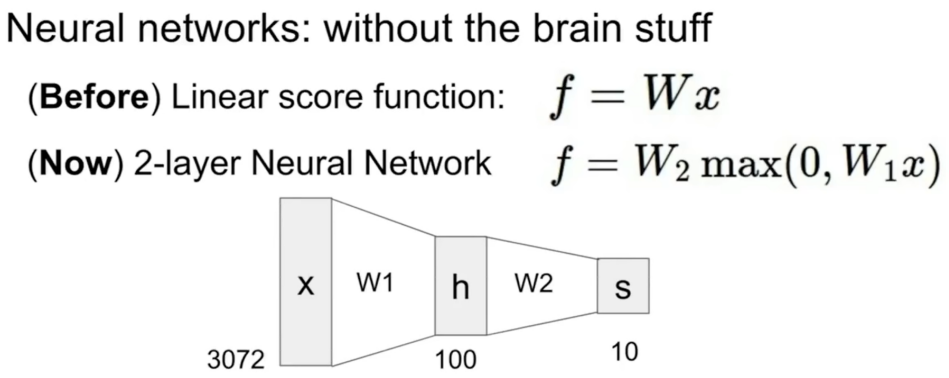
- **Add gate:** gradient distributor
- **Max gate:** gradient router
  only the maximum value will pass through the gate, and the others will be zero.
- **Mul gate:** gradient switcher

Additionally, if the element is a vector, the gradient will be Jacobian matrix with the same shape as the input.

For f(x, W) = Wx + b(x is a batch), the gradient is:

$$\nabla_x = dout * W^T, \quad \nabla_W = x^T * dout, \quad \nabla_b = \sum dout * 1 \tag{4.2}$$

4.2 Neural Networks:

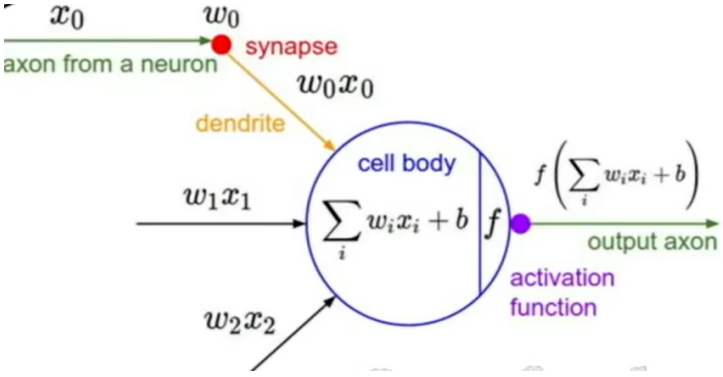


Neural Network Example: $f(x) = W_2 \max(0, W_1 x)$

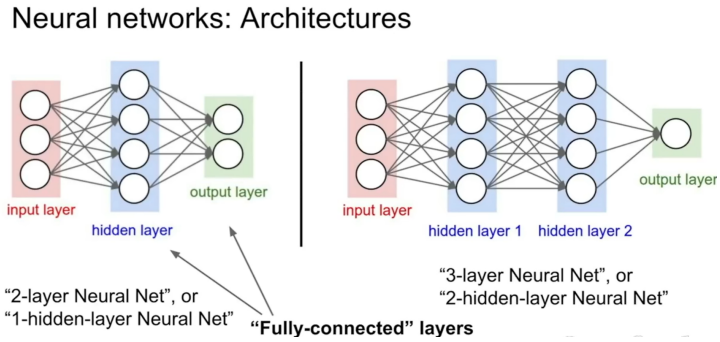
Here, W_1 contains the weights for the first layer, which can be thought of as computing the scores of the templates.

And W_2 contains the weights for the second layer, which can be thought of as computing the final scores for the classes by taking a weighted sum of the scores from the first layer.

Below are figures showing the structure of a neural network.



Neuron-like structure



Neural Network Architecture

Lecture 5: Convolutional Neural Networks

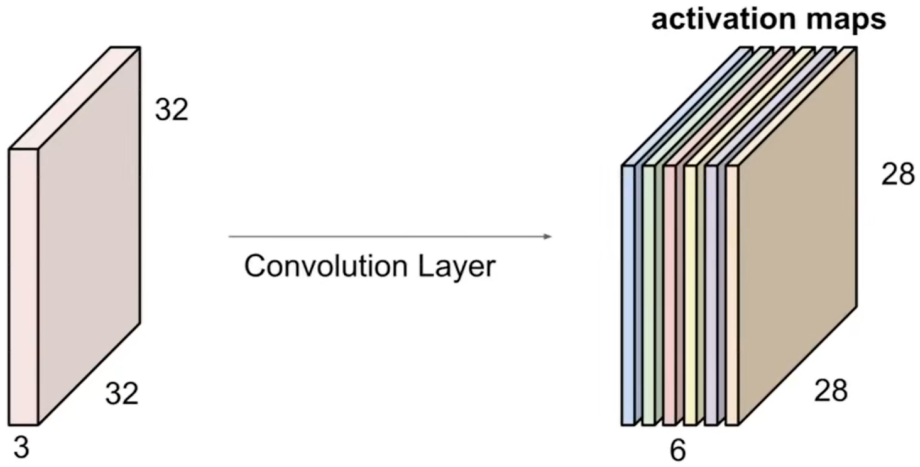
5.1 The Layers of CNNs

1. **Convolution Layer**

   Instead of stretching the input image into a long vector, now preserve the spatial structure.

   - **Filter:** Filters always extend the full **depth** of the input volume.

     Convolve(slide) over all spatial locations. Each filter is kind of looking for a specific feature in the image, so usually we have many filters in a convolutional layer.



Convolution Operation

   - **Convolution:**

     In mathematical terms, the convolution operation is defined as:

$$f[x,y] * g[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n1,n2] * g[x-n1, y-n2] \tag{5.1}$$

     which means that the filter is flipped both horizontally and vertically before being applied to the image.

     What we commonly do is a cross-correlation(互相关) operation, without flipping the filter, but we call it convolution.

Output size = $(W-F+2P)/S+1$, where W is the input size, F is the filter size, P is the padding size, and S is the stride size.

Dilated convolutions: a convolution operation where the filter is applied to the input with gaps between the filter elements, allowing for a larger receptive field without increasing the number of parameters.

2. **Pooling Layer**

   makes the representation smaller and more manageable.

   Similar to convolution layer, strides are also used in pooling layers, but padding is not commonly used.

   - **Max Pooling**
   - **General Pooling:** average pooling or even L2-norm pooling are also used.

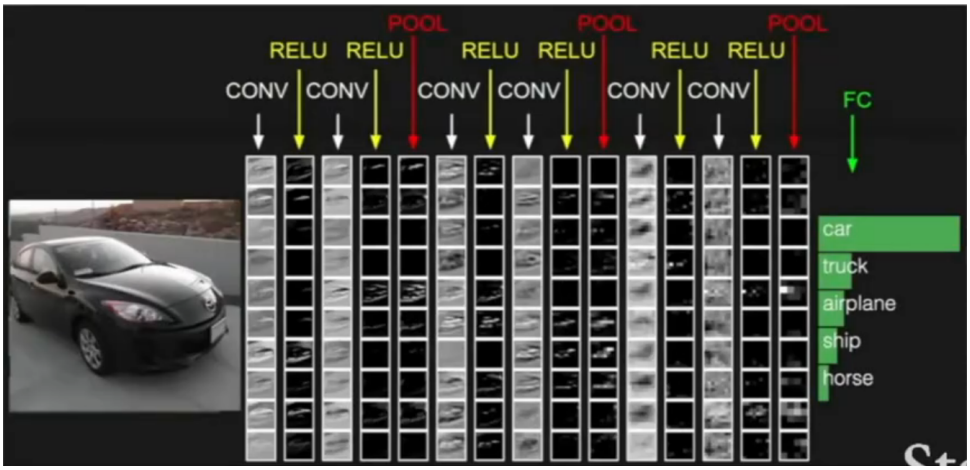
3. **Fully Connected Layer**

   At the end of the network, we have a fully connected layer, which is just a normal neural network layer.

4. **Normalization Layer**

   Many types of normalization layers have been proposed for use in ConvNet architectures, sometimes with the intentions of implementing inhibition schemes observed in the biological brain. However, these layers have since fallen out of favor because in practice their contribution has been shown to be minimal, if any.

In summary, the steps are like this:



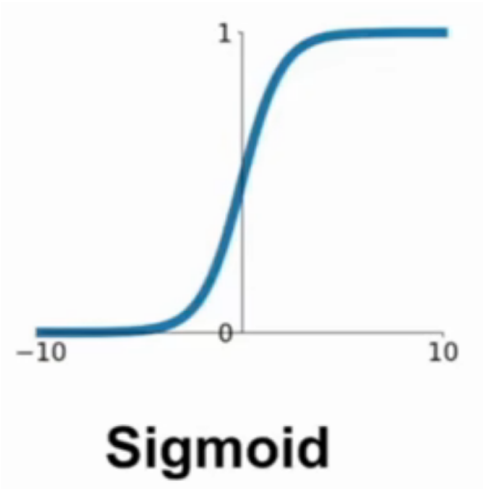
Steps of Convolutional Neural Networks

Lecture 6: Training Convolutional Neural Networks(Part 1)

6.1 Activation Functions

1. **Sigmoid:**

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{6.1}$$

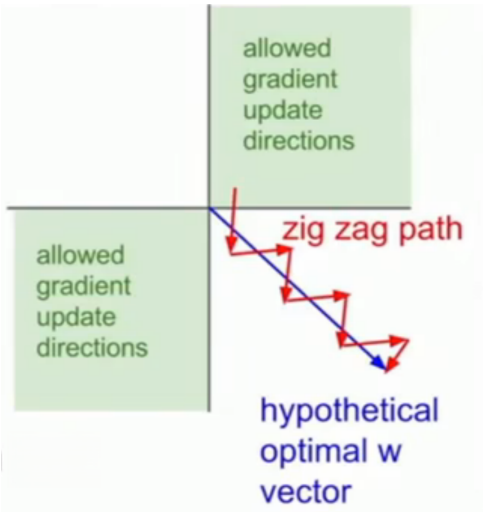


Sigmoid Activation Function

Three problems with sigmoid:

(a) Vanishing gradient problem: when the input is too large or too small, the gradient becomes very small, which slows down the training process.

(b) Output is not zero-centered: the output of the activation function is always positive, which means the gradients($\partial L/\partial \omega = \partial L/\partial f \cdot \partial f/\partial \omega = \delta \cdot x$, in which f is the local gradient including the activation function) are always positive or negative according to $\partial L/\partial \omega$, which can lead to inefficient updates:



Sigmoid Output is not Zero-Centered

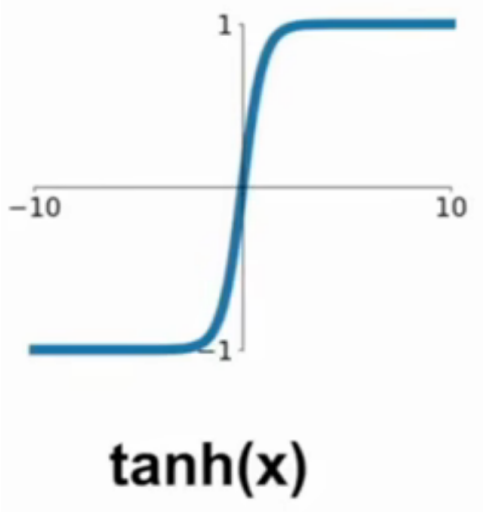
For example, if the optimal direction is the blue arrow, since positive or negative gradients only, the zig zag path is taken, which is inefficient.

(c) exp() is computationally expensive.

2. **tanh:**

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{2}{1 + e^{-2x}} - 1 \tag{6.2}$$

It gets rid of the not zero-centered problem compared to sigmoid, but still has the vanishing

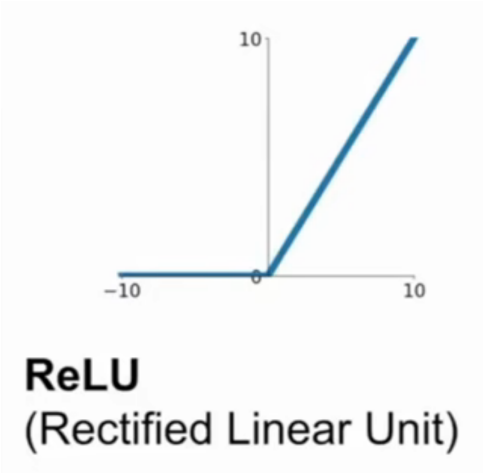


Tanh Activation Function

gradient problem.

3. **ReLU(Rectified Linear Unit):**

$$\text{ReLU}(x) = \max(0, x) \tag{6.3}$$



ReLU Activation Function

- Does not saturate(gradient equals 0) in the positive domain, so it does not suffer from the vanishing gradient problem.
- Computationally efficient: only requires a simple thresholding at zero.
- Converges much faster than sigmoid or tanh.
- More biologically plausible: it is similar to the firing rate of a neuron, which is zero when the input is negative and increases linearly with the input when it is positive.

However, it is still not zero-centered.

Dead ReLU: when the input is negative, the output is always zero, which means the gradient is always zero, and the neuron is "dead": never update.

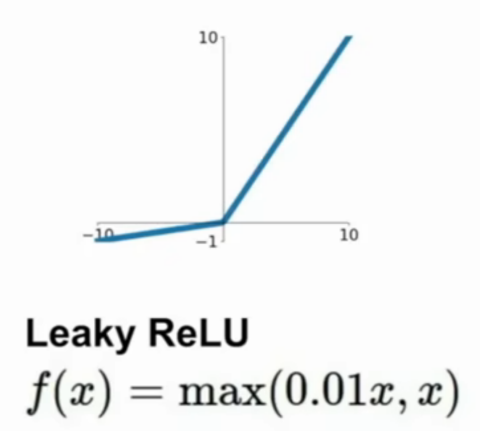
People like to initialize ReLU with a small positive bias(0), so that the neuron is not dead at the beginning.

4. **Leaky ReLU:**

   Parametric ReLU:

$$\text{Parametric ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \tag{6.4}$$

   where α is a small positive constant, usually set to 0.01.



Leaky ReLU Activation Function

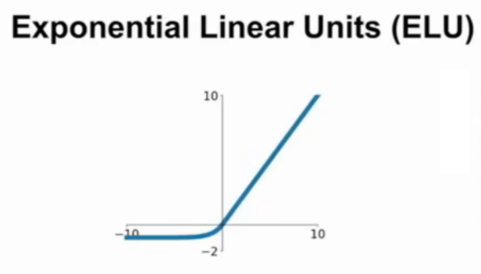
   On the base of ReLU:

   - it does not saturate.
   - it will not die.

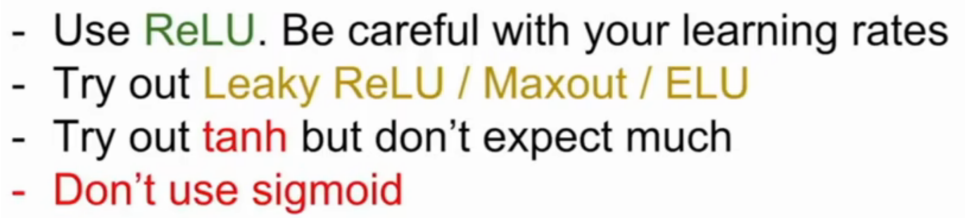
5. **Exponential Linear Unit (ELU):**

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases} \tag{6.5}$$

   - All benifits of ReLU.
   - Cloer to zero mean output.
   - Negative saturation region: add some robustness to noise.



ELU Activation Function



Suggestions for Choosing Activation Functions

6. **Maxout neuron:**

$$\text{Maxout}(x) = \max(\omega_1^T x + b_1, \omega_2^T x + b_2) \tag{6.6}$$

Linear Regime. Does not saturate. Does not die.

However, double the number of parameters compared to ReLU.

Some suggestions for choosing activation functions:

6.2 Data Preprocessing

1. **Zero centering**: For [32, 32, 3] images:
   - Subtract the mean image(AlexNet): [32, 32, 3].
   - Subtract the mean of each channel(VGGNet): mean of each 3 channels.
2. **Normalization:**For image data, we accutualy only do zero centering, since the pixel values are already in the range of 0 to 255, which is not too large.

6.3 Weight Initialization

1. **Zero initialization:** all neuron will be the same, and the gradients will be the same, so all neurons will learn the same features, which is not good.
2. **Small random initialization:** usually use a Gaussian distribution with mean 0 and small standard deviation, e.g., 0.01.

   The problem is in deep networks, Wx becomes smaller and smaller, all the activation values will shrink to zero, then the gradients will also shrink to zero.
3. **Normal Gaussian Initialization:** problem: almost all the neurons completely saturate, so the gradients are almost zero.

4. **Xavier Initialization:** for tanh and sigmoid, we want the variance of the output to be the same as the variance of the input, so we set the variance of the weights to be $1/n$, where n is the number of input neurons.

6.4 Batch Normalization

1. compute the **empirical** mean and variance independently for each dimension. not computed based on each batch.

2. normalize:

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{x(k)}]}{\sqrt{Var[x^{(k)}]}} \tag{6.7}$$

   Usually inserted after Fully Connected or Convolutional layers, before the activation function.

3. scale and shift:

$$y^{(k)} = \gamma \hat{x}^{(k)} + \beta \tag{6.8}$$

   where γ and β are learnable parameters, which

   - improves gradient flow through the network.
   - allows higher learning rates.
   - reduces the strong dependence on initialization.
   - acts as a regularizer, reducing the need for Dropout.

6.5 babysitting the learning process

1. Preprocess the data

2. Choose the architecture

6.6 Hyperparameter Optimization

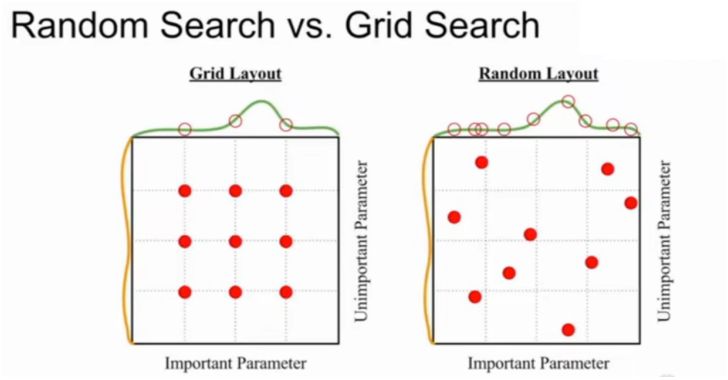
learning rate: 1e-5 1e-3
consider using log space, but uniform space.

1. first stage: only a few epochs to get rough idea of what params work.

2. second stage: lobger running time, finer search.

   when the cost get 3x larger than the original cost, break out.

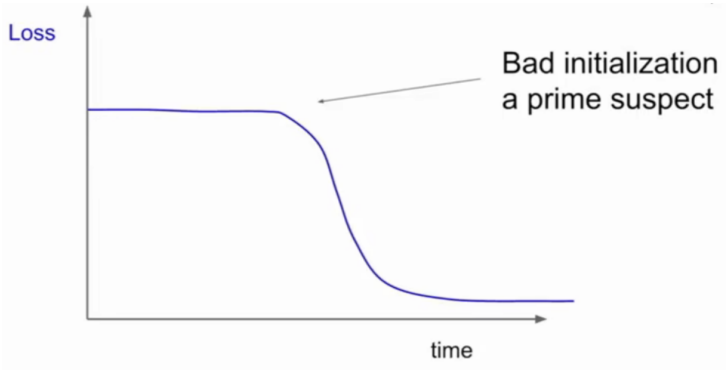
Consider random search instead of grid search, since it is more efficient and can cover a larger space.



Hyperparameter Search

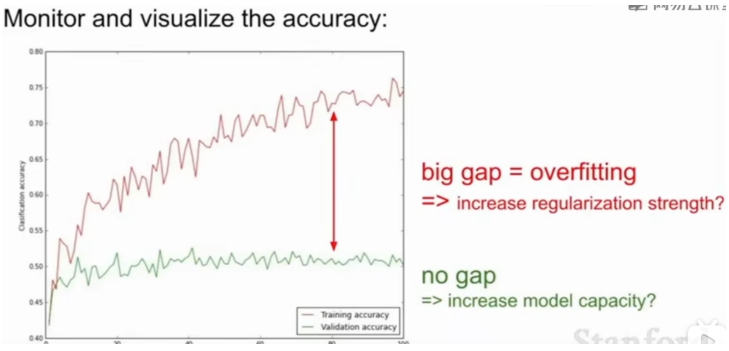
Some expeirence:

- bad initialization.



loss curve is flat then suddenly decreasing

- overfit or underfit.



Overfitting and Underfitting

- ratio between the updates and values: around 0.001.

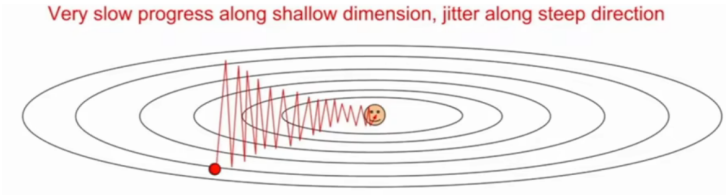
Lecture 7: Training Convolutional Neural Networks(Part 2)

7.1 Optimization Algorithms

1. Problems with SGD

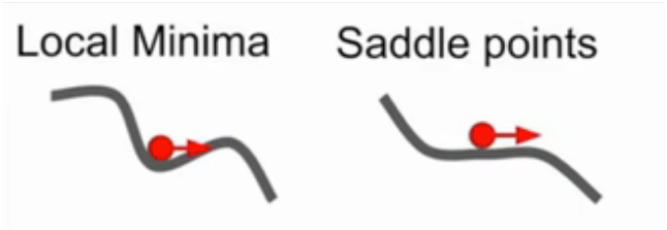
(a) Poor conditioning: Follow a zig-zag path to convergence, reducing efficiency.

Poor conditioning essentially happens when the loss function has very different curvature in different directions (the Hessian's largest and smallest eigenvalues differ greatly, i.e., high condition number)



p1

(b) saddle point/local minima



Saddle Point

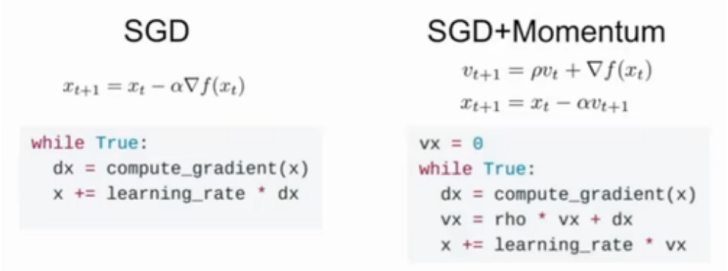
(c) noisy gradients

every time only a small batch is used to compute the gradient other than the full training dataset, leading to high variance in the updates.

2. SGD + Momentum

- SGD + Momentum

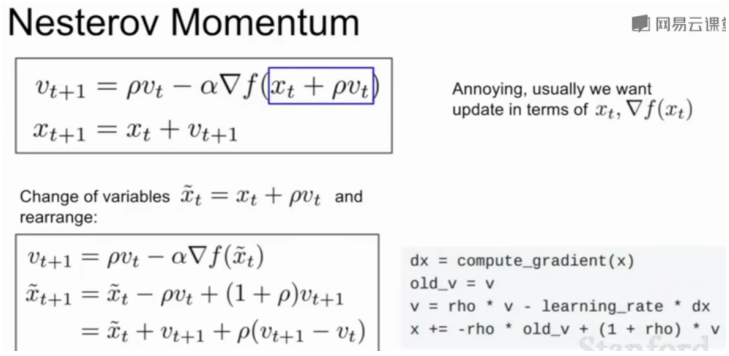
  build up "velocity" as mean of gradients with weight decay:



SGD with Momentum

- Nesterov Momentum

  Nesterov momentum is a variant of momentum that incorporates the gradient of the loss function at the "lookahead" position, which can lead to better convergence properties.



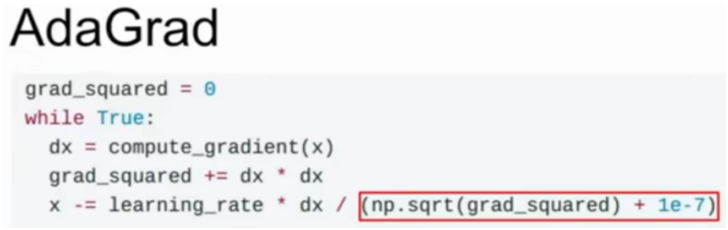
Nesterov Momentum

3. Adagrad and RMSprop

   - Adagrad

     Adagrad is an adaptive learning rate optimization algorithm that adjusts the learning rate for each parameter based on the historical gradients. It gives larger updates for infrequent parameters and smaller updates for frequent parameters.

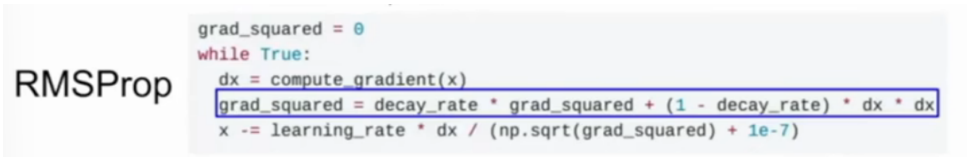
     However, the learning rate shrinks continuously, which can lead to premature convergence. Therefore, it is often not used in practice.



Adagrad

   - RMSprop

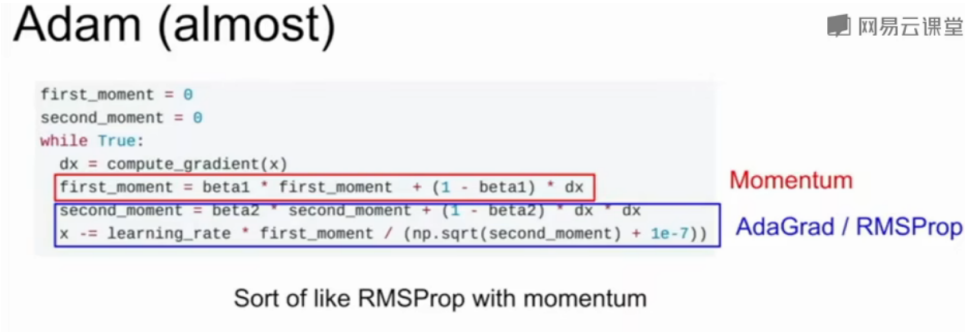
     Based on Adagrad, RMSprop maintains a moving average of the squared gradients to normalize the gradients. This helps to mitigate the problem of vanishing or exploding gradients.



RMSprop

4. Adam

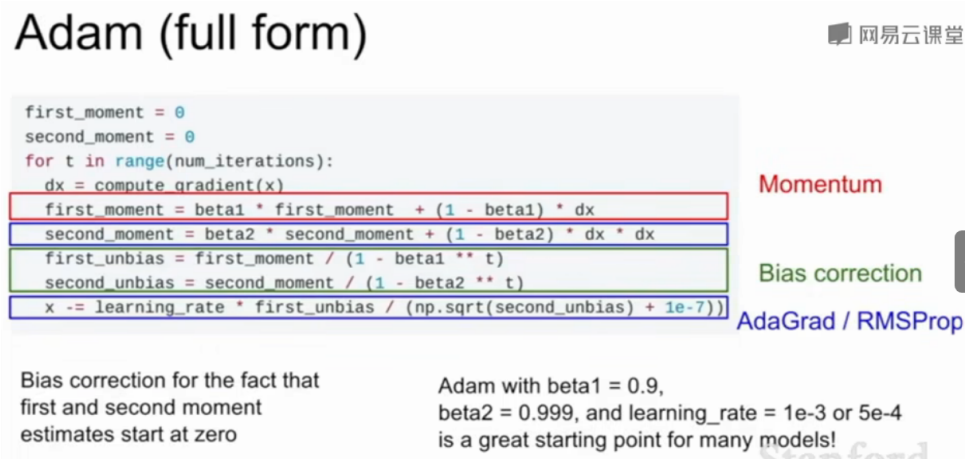
   Adam (Adaptive Moment Estimation) is an optimization algorithm that combines the benefits of both Adagrad and RMSprop. It maintains a moving average of both the gradients and the squared gradients, which helps to adaptively adjust the learning rates for each parameter.



Adam

Since initializing the second moment to 0, the first step can be very large, which can lead to instability in the optimization process. To counteract this, Adam includes a bias correction term that adjusts the learning rates based on the moving averages of the gradients and squared gradients.

Hence bias is used:



Adam with Bias Correction

7.2 Second-Order Optimization

Second-order optimization methods use information about the curvature of the loss function to make more informed updates to the model parameters.

L-BFGS: need to compute the Hessian matrix, which can be expensive for large models.

7.3 Model Ensembles

To reduce training error, combines several models to improve overall performance.

7.4 Regularization

1. Dropout:

   In each forward pass, randomly set some neurons (activation) to zero with a probability p, which is a hyperparameter.

   - Add randomnessForce the model to learn more robust features that are useful in conjunction with other neurons, rather than relying on specific ones.

- average out the randomness at test-time at test time

2. Batch Normalization
3. Data Augmentation:

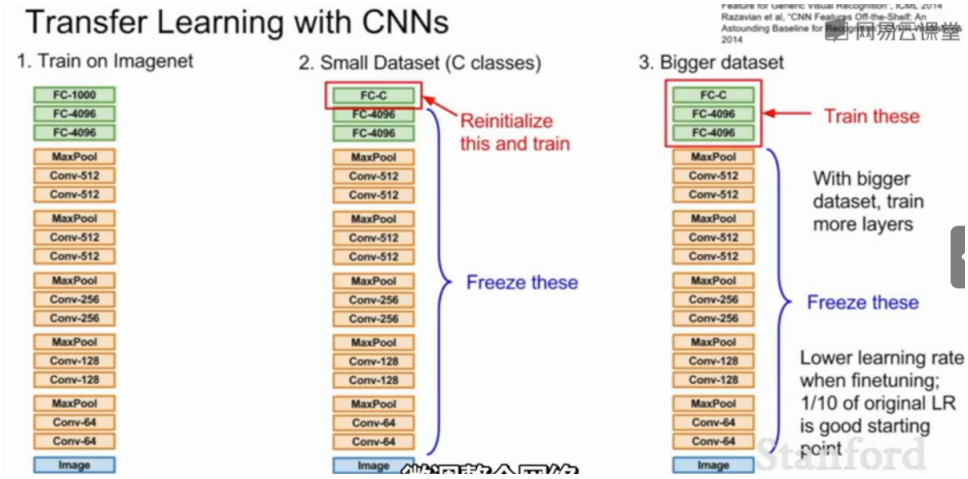
   Random mix of translation/rotation/stretching/shearing/lens distortions...

4. Fracional Max Pooling
5. Stochastic Depth:

   Randomly drop entire layers during training, which helps to prevent overfitting and encourages the model to learn more robust features.

7.5 Transfer Learning

Transfer learning is a technique where a model trained on one task is fine-tuned on a different but related task.



Transfer Learning

Lecture 8

8.1 Deep Learning Framework

1. Pytorch

   Every time build a computational graph, and then compute the gradients using backpropagation(Dynamic Computational Graph).

   Use a different data type(e.g. torch.cuda.FloatTensor) to adapt to GPU.

   - Variable is a node in a computational graph. requires_grad=True is used to track gradients.
   - x.data is a Tensor.
   - x.grad is a Variable of gradients.
   - x.grad.data is a Tensor of gradients.

   Use nn to encapsulate layers and models.

   Visdom: something like TensorBoard.

2. TensorFlow

   - Build computational graphs first(Static Computational Graph). TensorFlow Fold allows for dynamic computation graphs.
   - Automatic differentiation.

   Keras: High-Level Wrapper

   TensorBoard: Visualization tool for TensorFlow.